

<https://forms.gle/vy2Jj154TJeo1Hat7>



CS 498: Paged Attention

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Recall: Continuous batching and Chunked-Prefill



**Continuous
Batching**
(Orca)

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Recall: Continuous batching and Chunked-Prefill

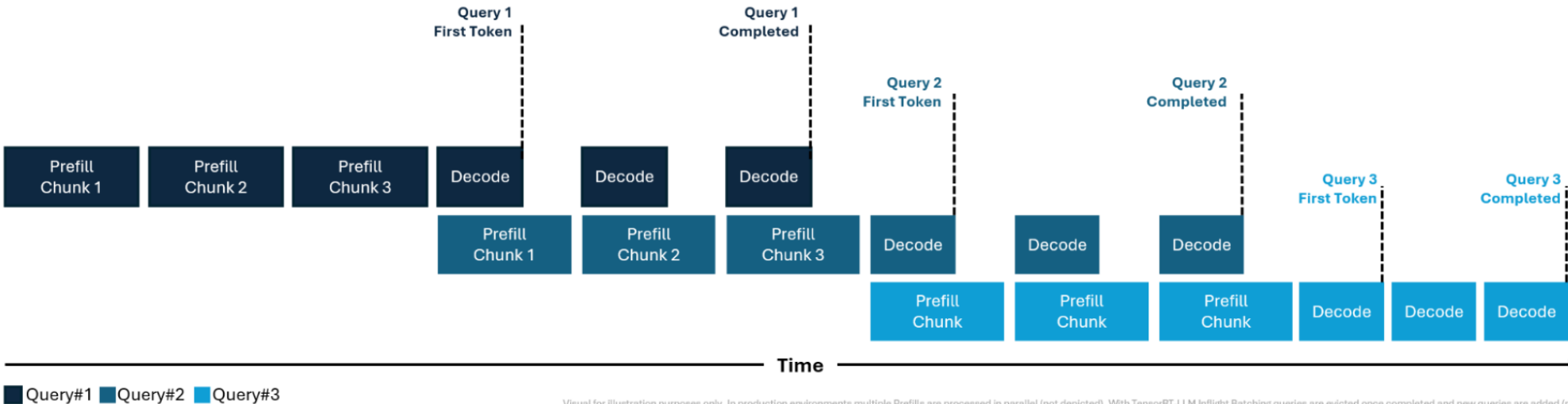


Continuous
Batching
(Orca)

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Chunked-
Prefill

W/ TensorRT-LLM Chunked Prefill



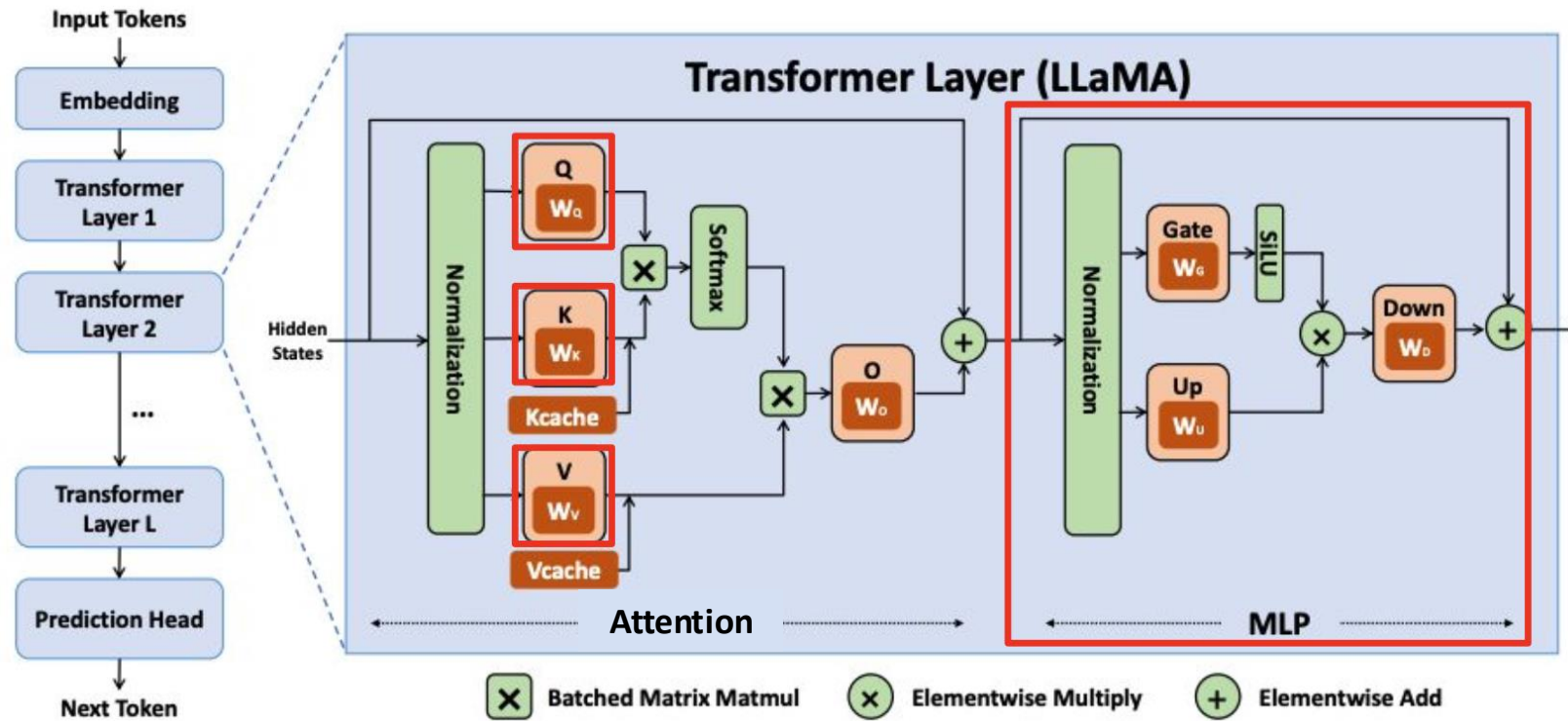
Visual for illustration purposes only. In production environments multiple Prefills are processed in parallel (not depicted). With TensorRT-LLM Inflight Batching queries are evicted once completed and new queries are added (not d

LLM Inference with Large Batch Sizes

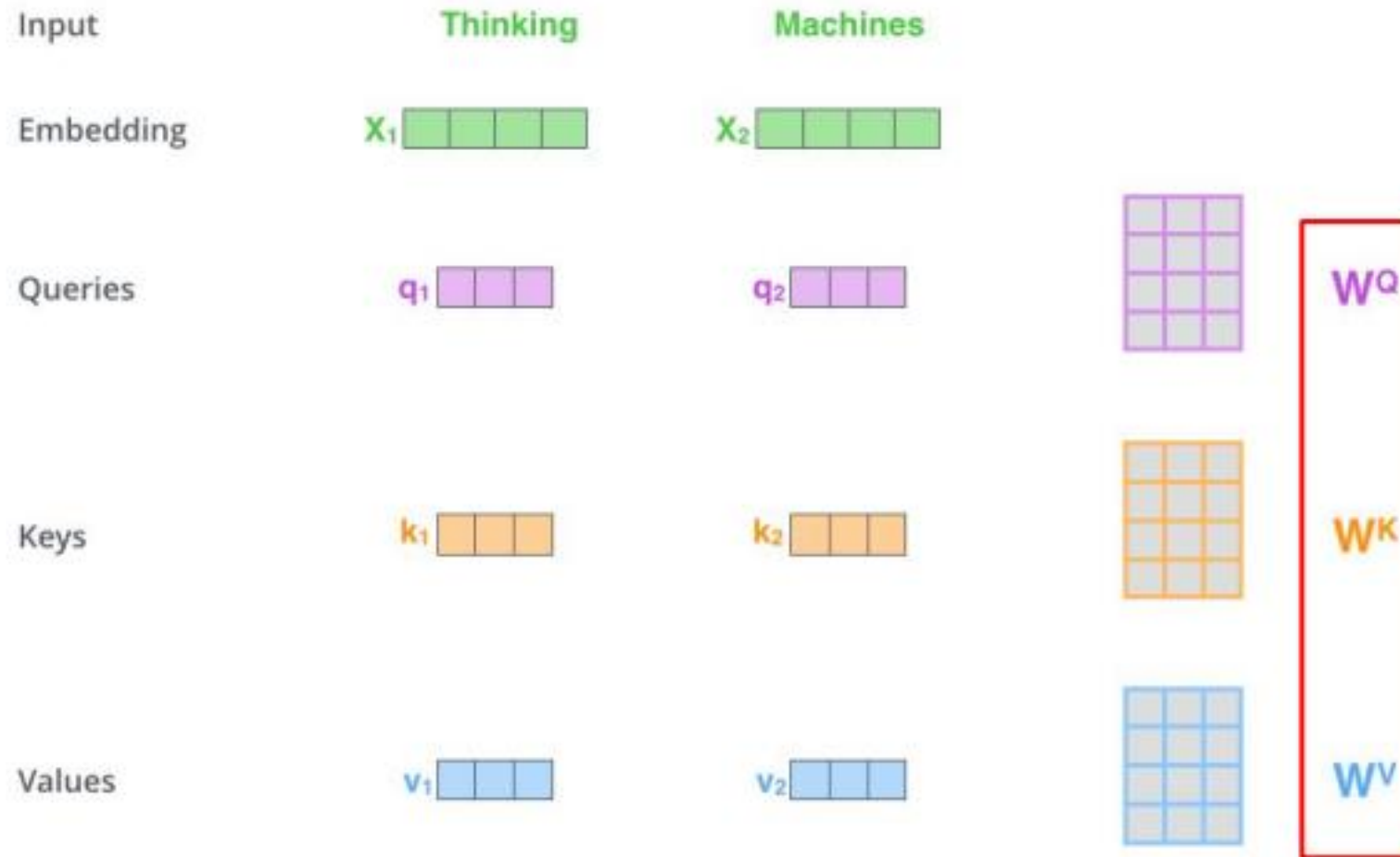
- Paged Attention

Objective: Learn why KV cache is important for LLM inference and how to manage KV cache efficiently

Self-Attention in Transformers

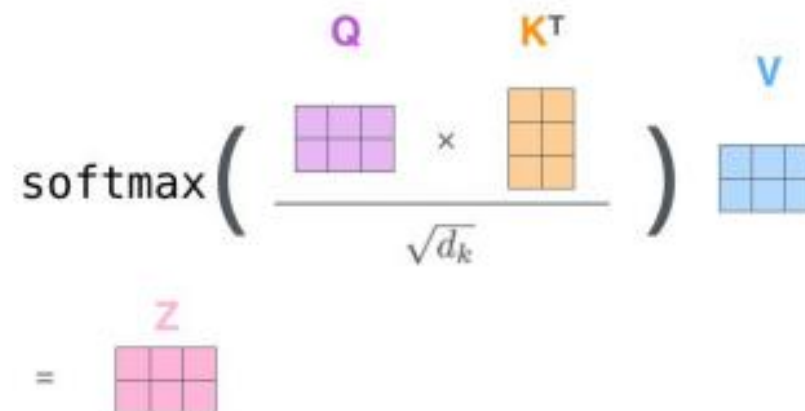


Self-Attention

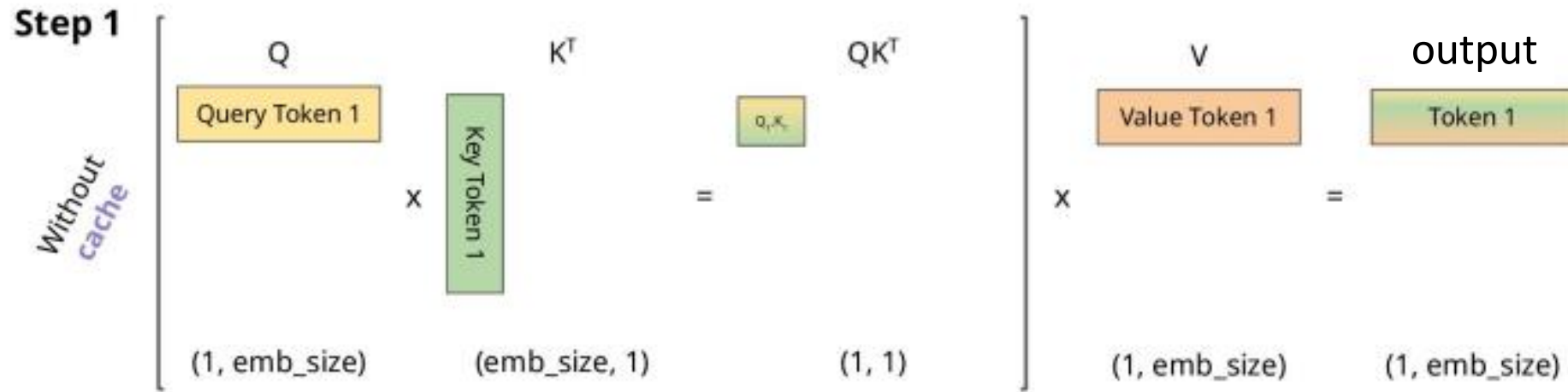




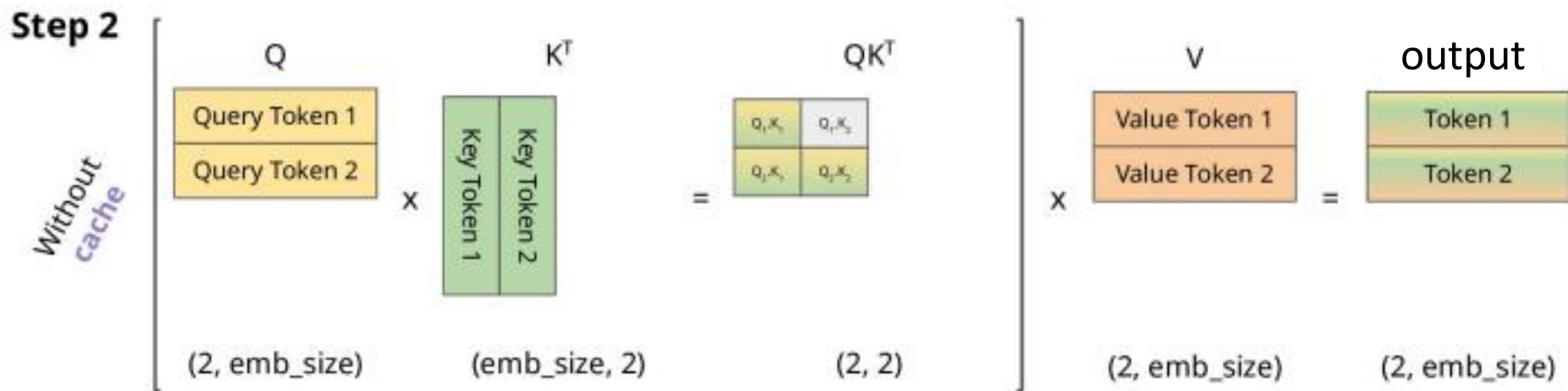
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



Self-Attention with KV Cache

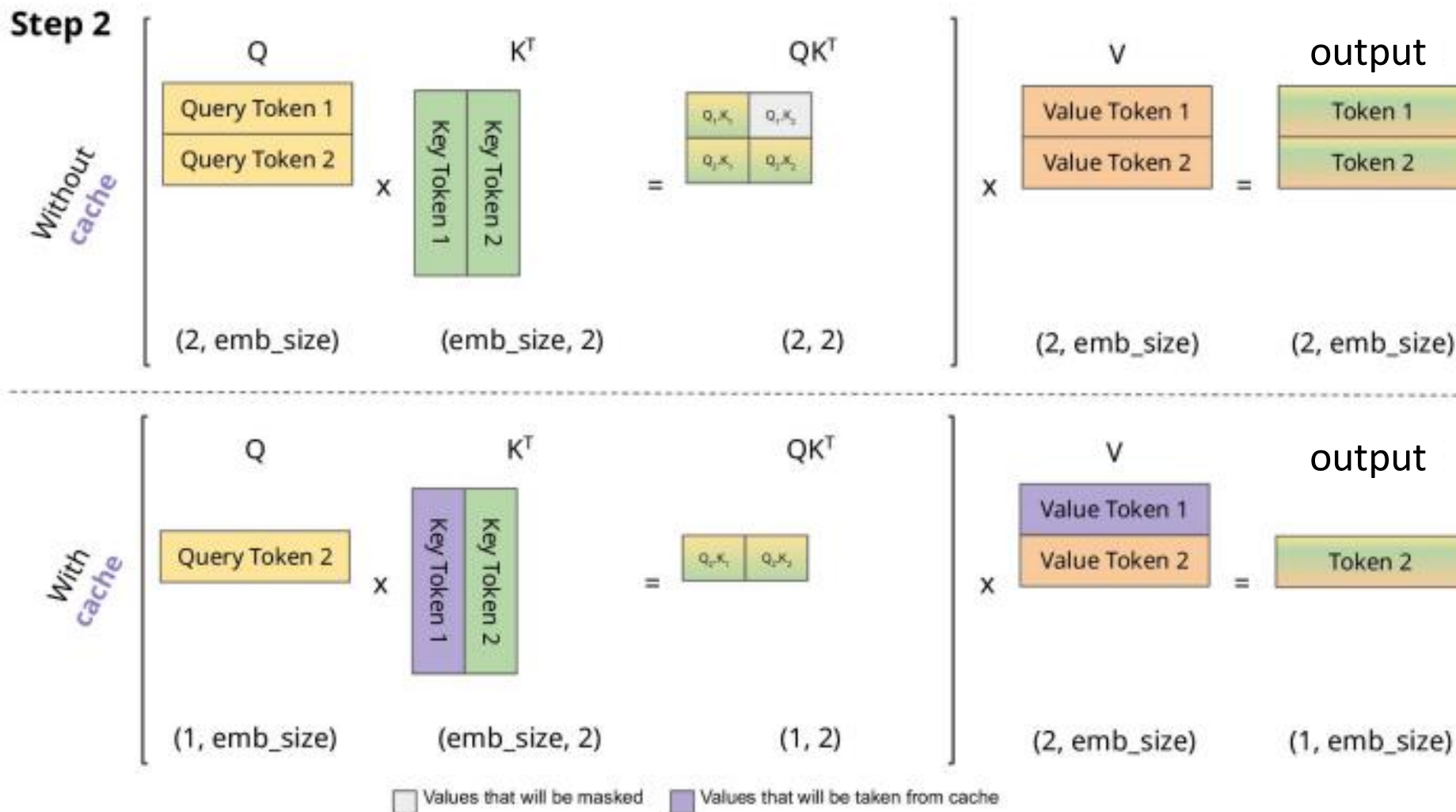


Self-Attention with KV Cache

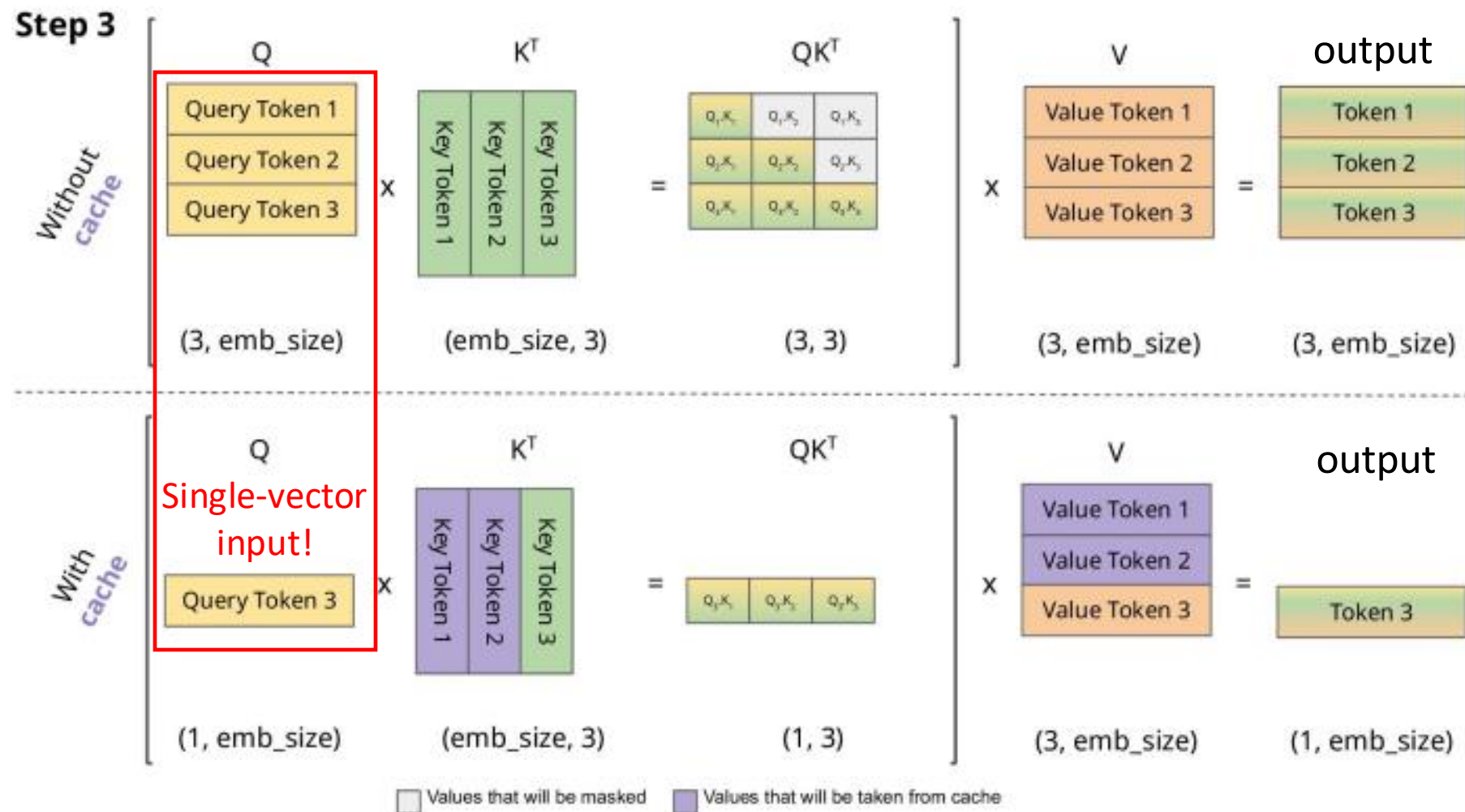


□ Values that will be masked □ Values that will be taken from cache

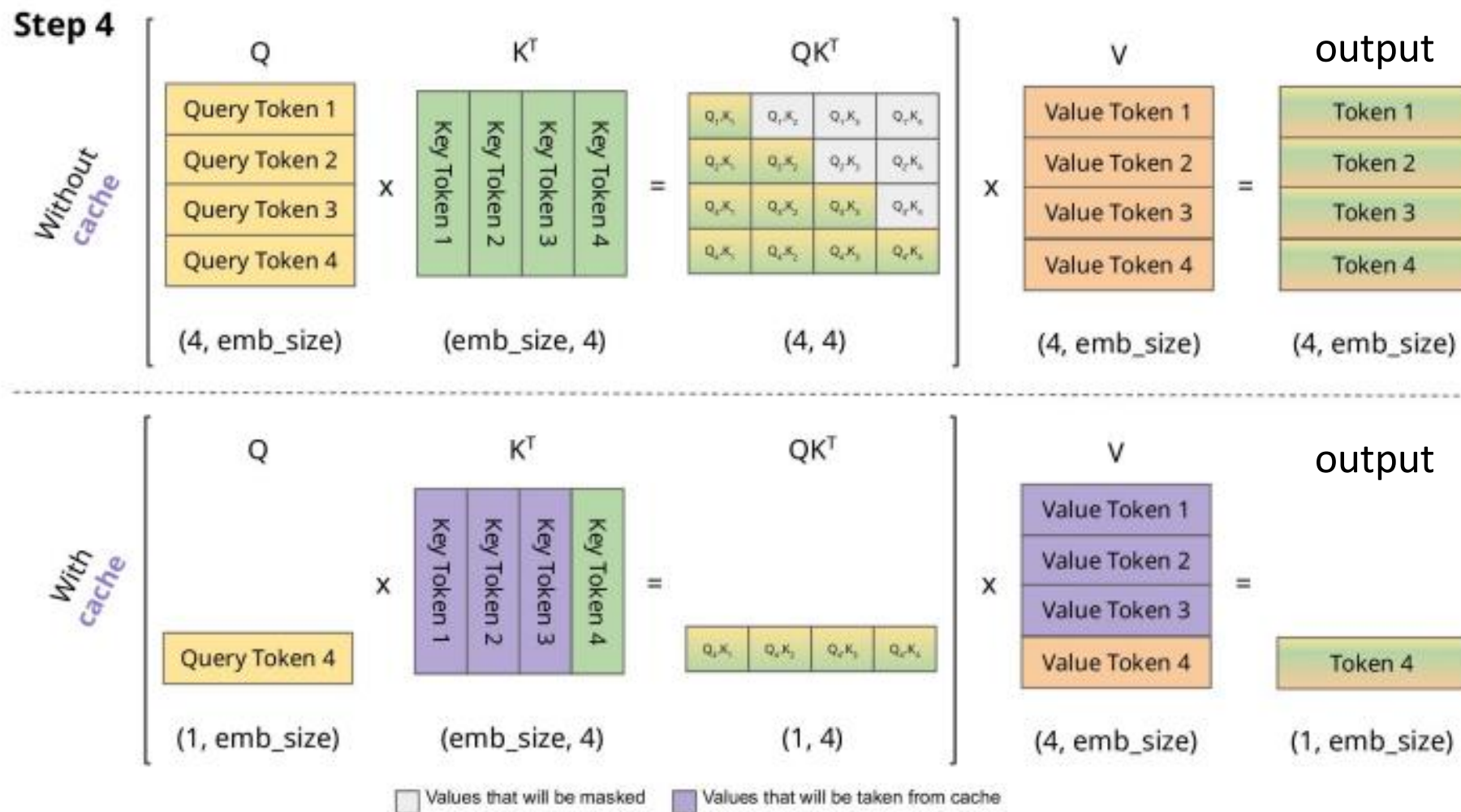
Self-Attention with KV Cache



Self-Attention with KV Cache



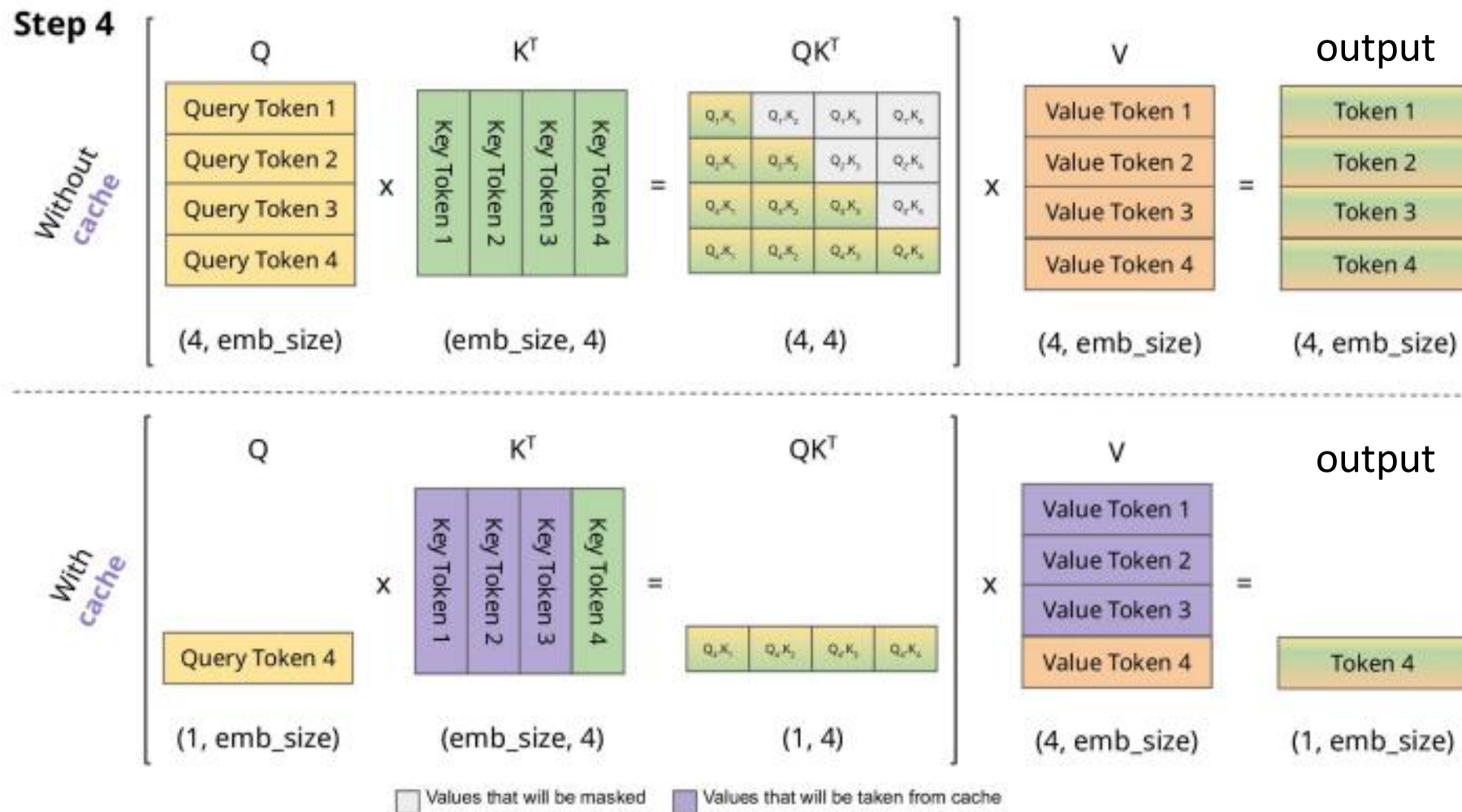
Self-Attention with KV Cache



Self-Attention with KV Cache



S: sequence length
H: hidden dim size

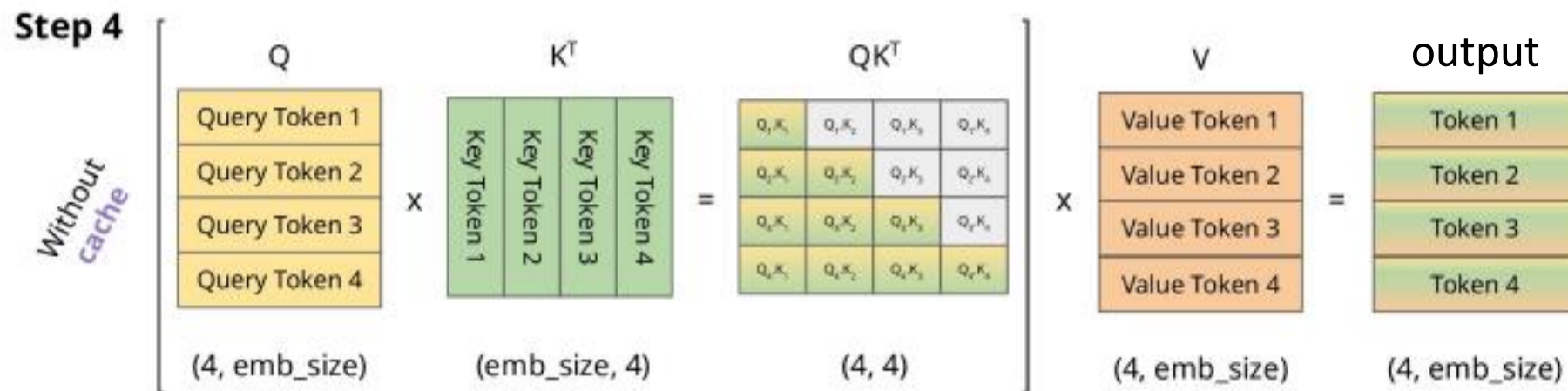


Question: Time and memory complexity without and with cache?

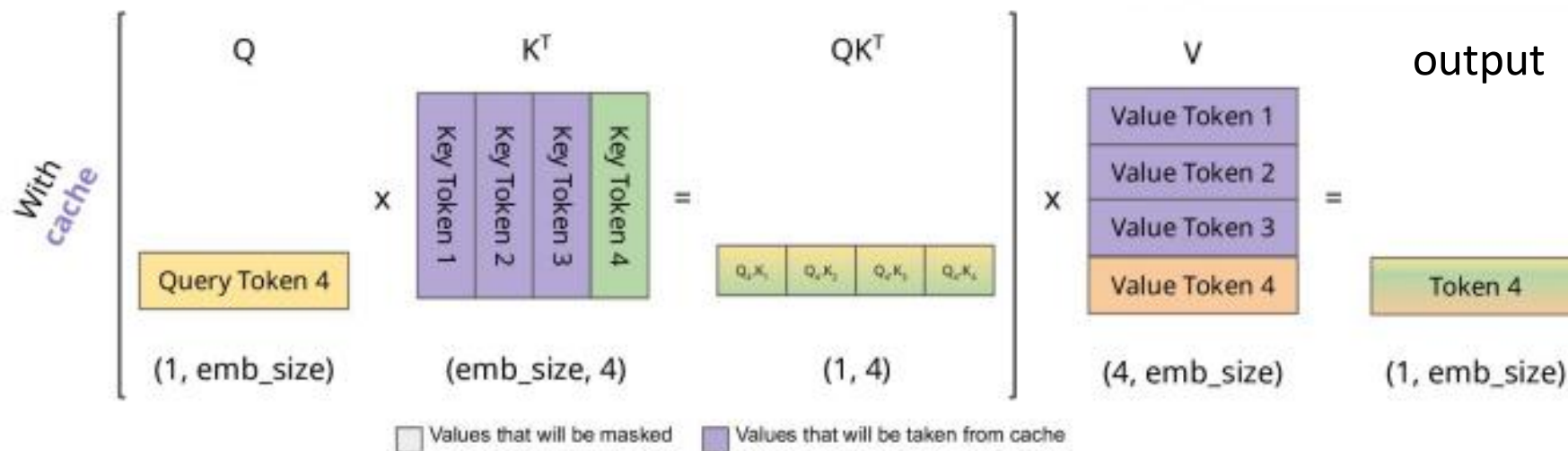
Self-Attention with KV Cache



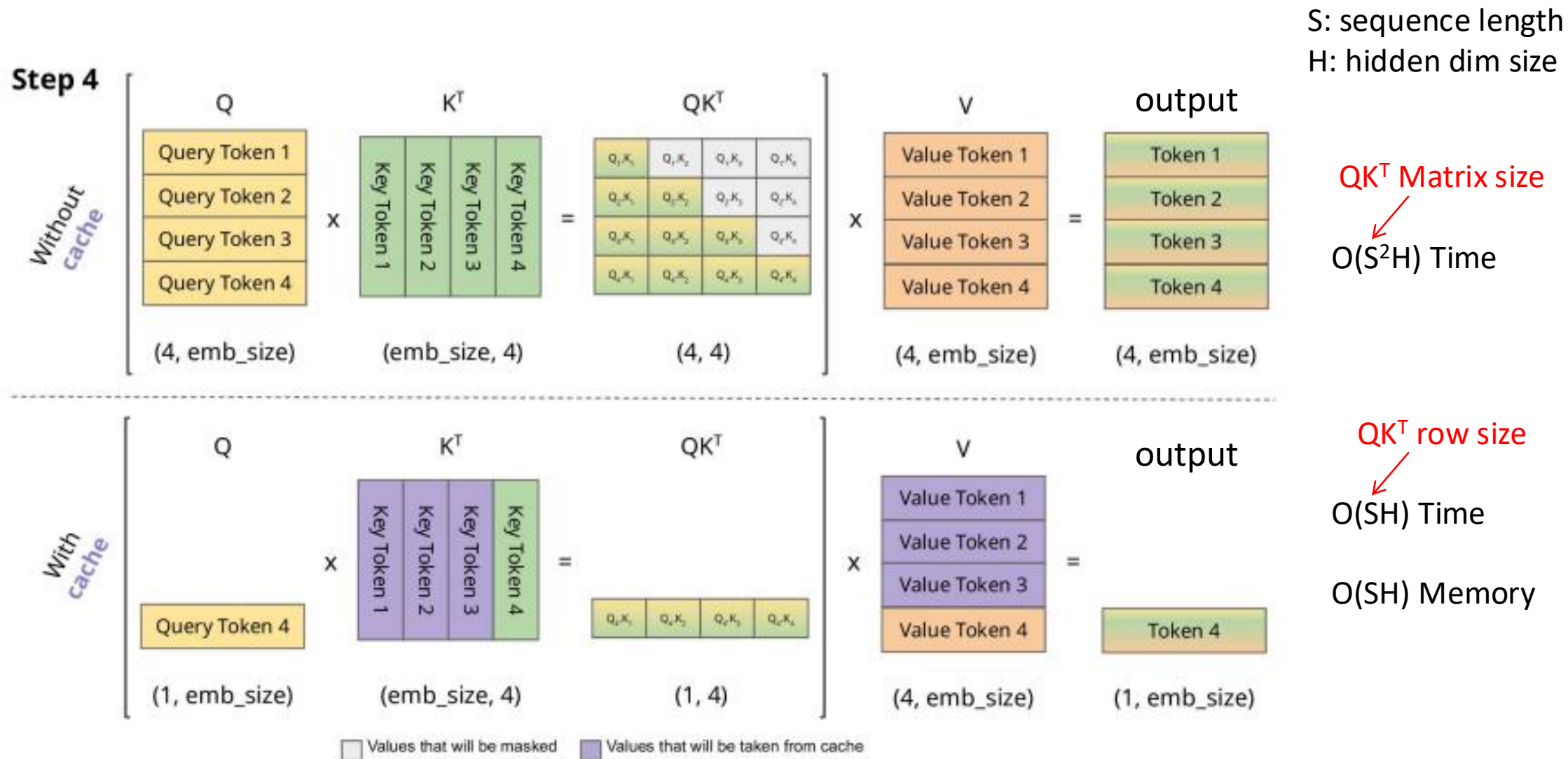
S: sequence length
H: hidden dim size



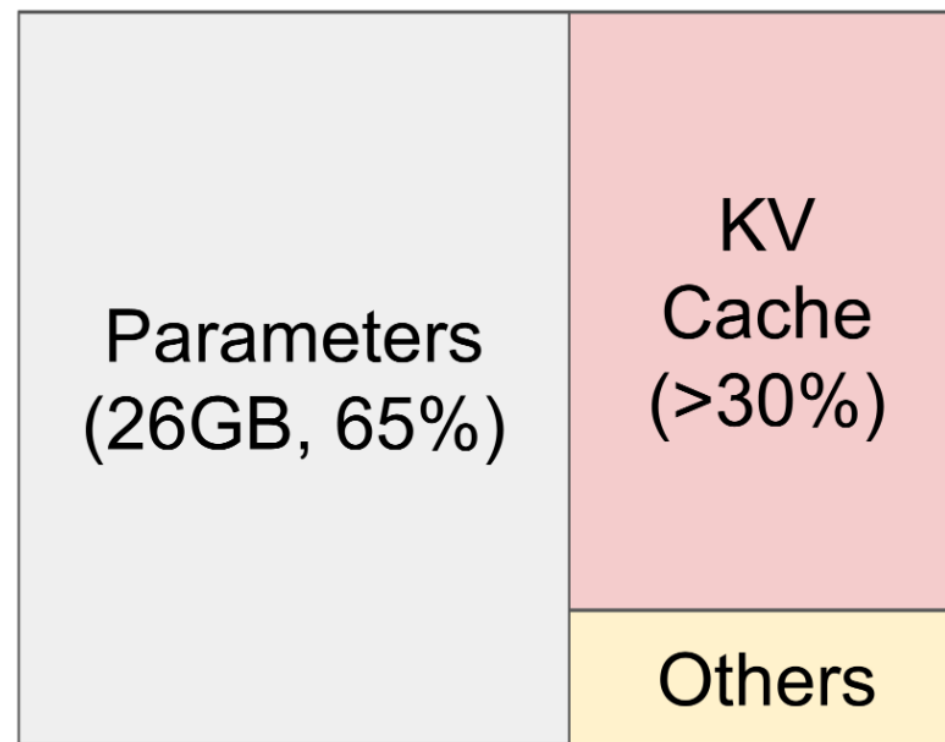
QK^T Matrix size
↓
O(S²H) Time



Self-Attention with KV Cache

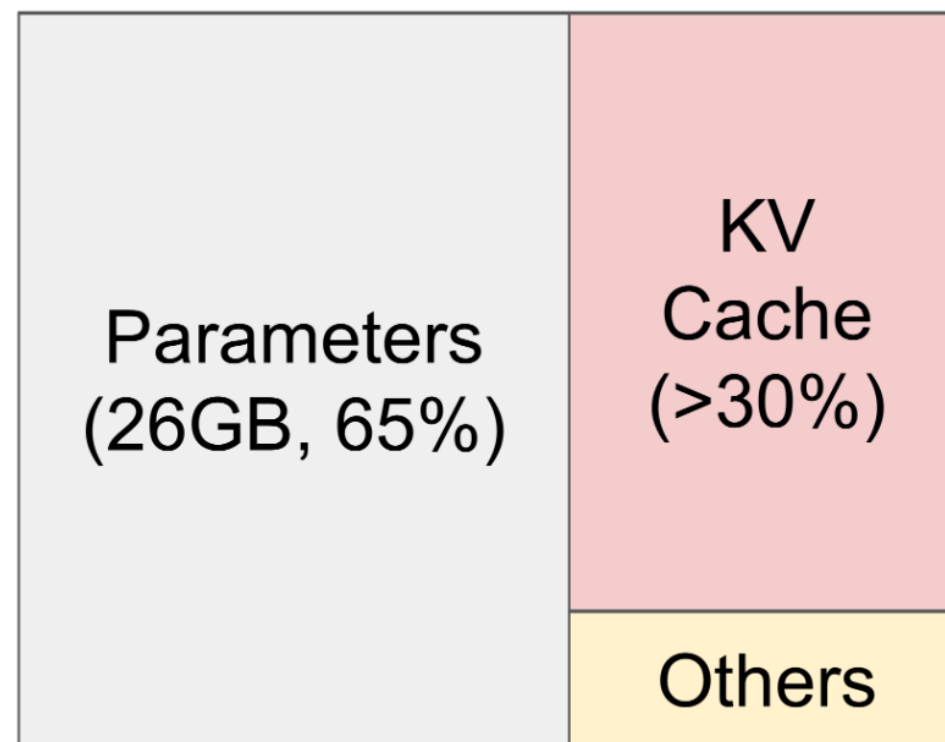


- KV cache
 - Sequence length
 - # Hidden
 - # Layers
 - Batch size



LLaMA-13B on A100 40 GB

- KV cache
 - Sequence length
 - # Hidden
 - # Layers
 - Batch size

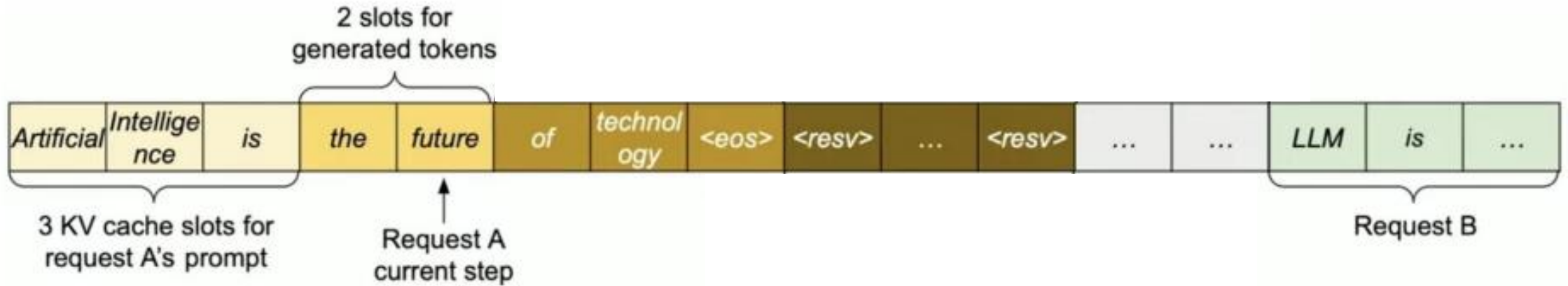


LLaMA-13B on A100 40 GB

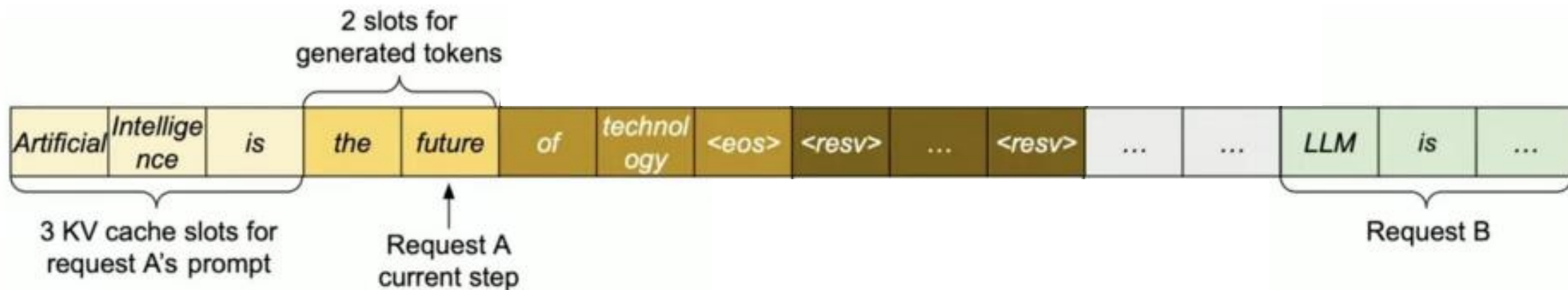
Motivation: Memory Waste in KV Cache



How about we allocate some contiguous memory per request:
 $\text{max-seq-len} * \text{hidden-dim} * \text{num-layers}$?

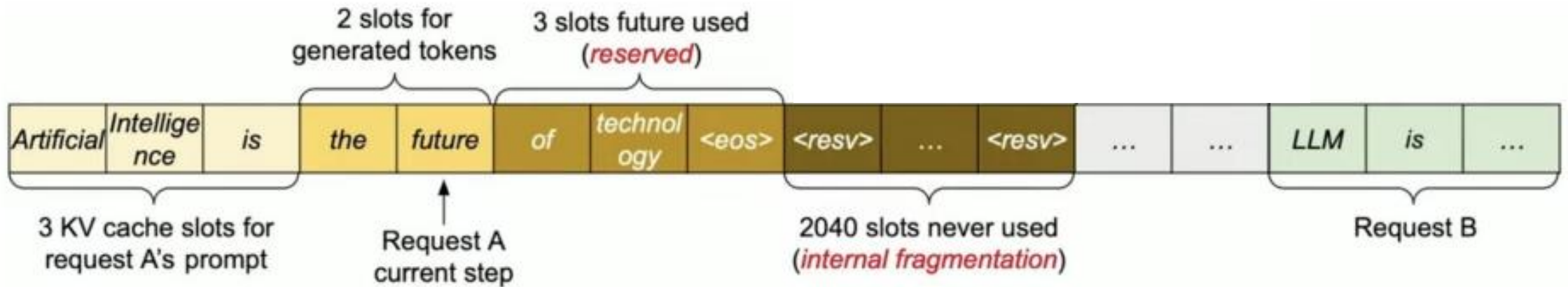


Motivation: Memory Waste in KV Cache



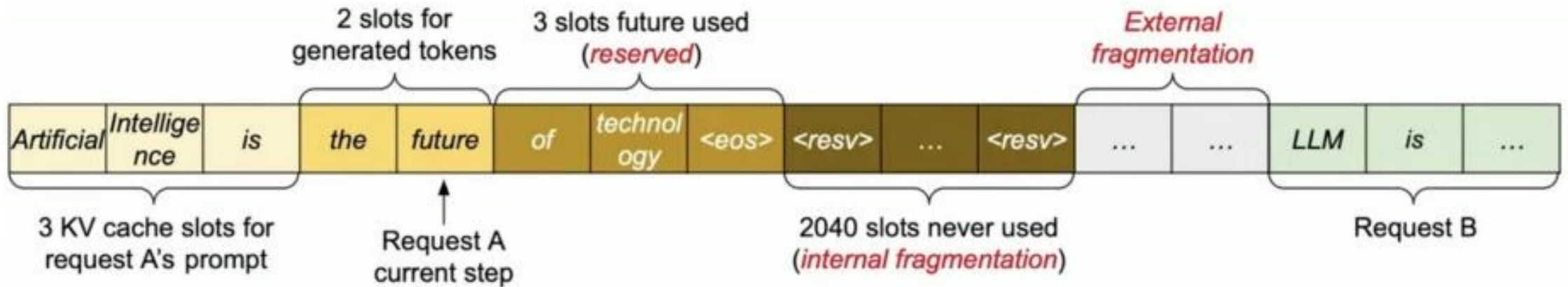
Question: Where is memory being wasted?

Motivation: Memory Waste in KV Cache



- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected
- **Reservation:** memory currently unused but reserved for future use by the same request

Motivation: Memory Waste in KV Cache

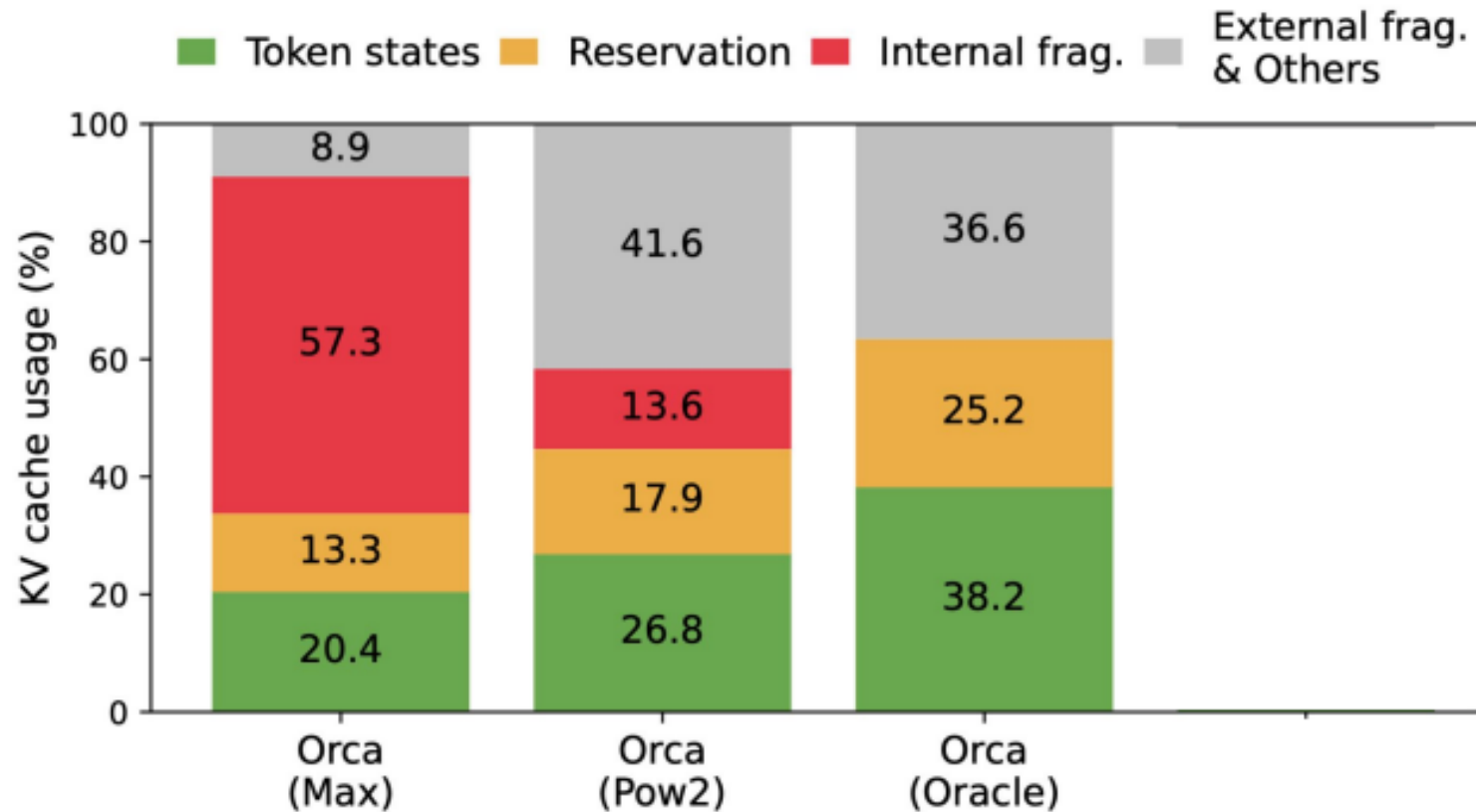


- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected
- **Reservation:** memory currently unused but reserved for future use by the same request
- **External fragmentation:** occurs when fixed-size memory blocks do not match the variable lengths of sequences, leaving unused gaps between them that cannot be allocated to other processes

Motivation: Memory Waste in KV cache



- Only 20-40% of KV cache is used to store the actual token states



- PagedAttention algorithm allows for storing attention key-value pairs in **non-contiguous blocks** in the memory



Key and value vectors

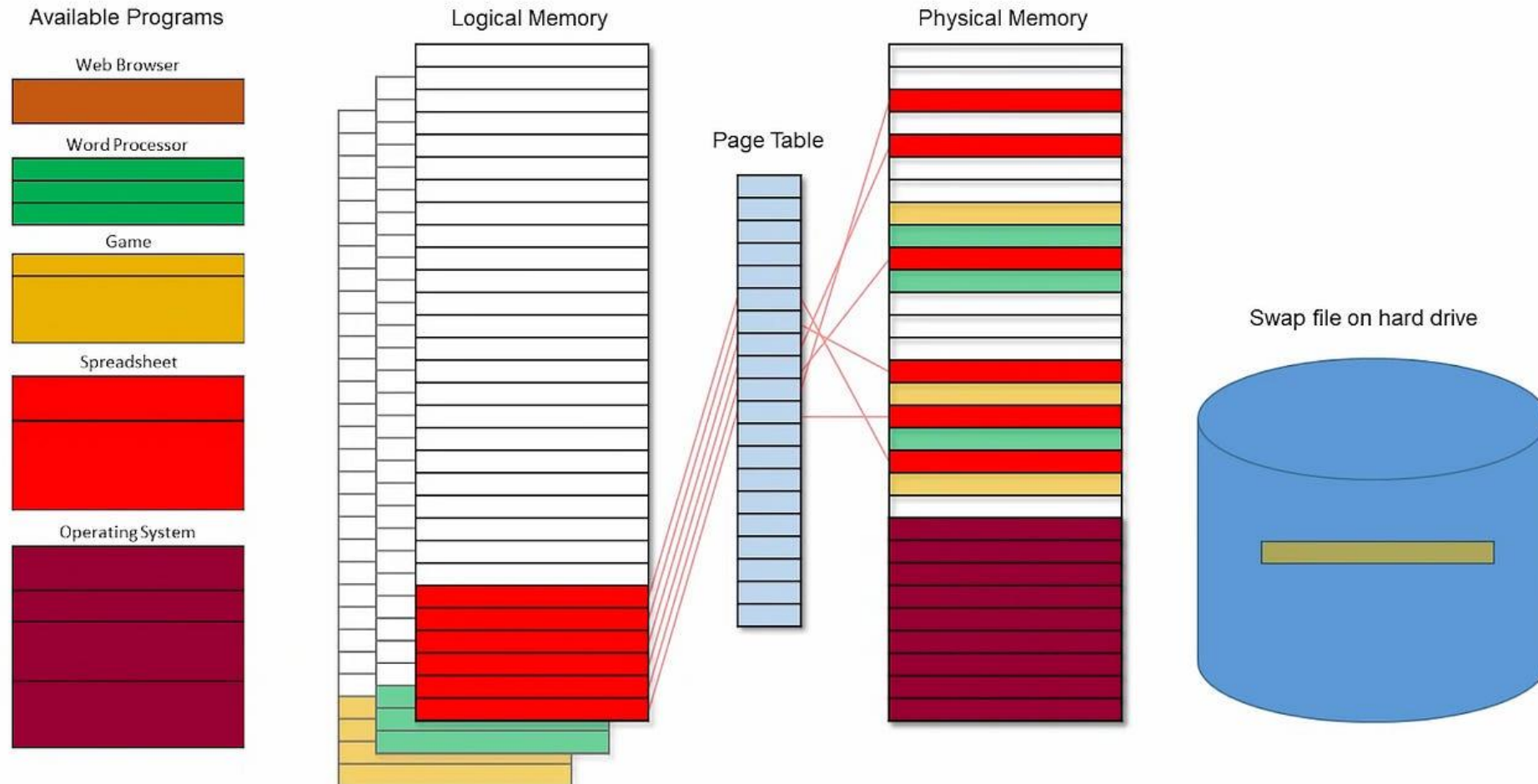
Block 1	years	ago	our	fathers
Block 2	brought	forth		
Block 0	Four	score	and	seven

block size = 4

Inspiration: Memory Paging (Look-up Table) in OS



Paged Memory



- PagedAttention algorithm allows for storing attention key-value pairs in **non-contiguous blocks** in the memory
- Each KV block is a fixed-size contiguous chunk of memory that stores KV cache from left to right

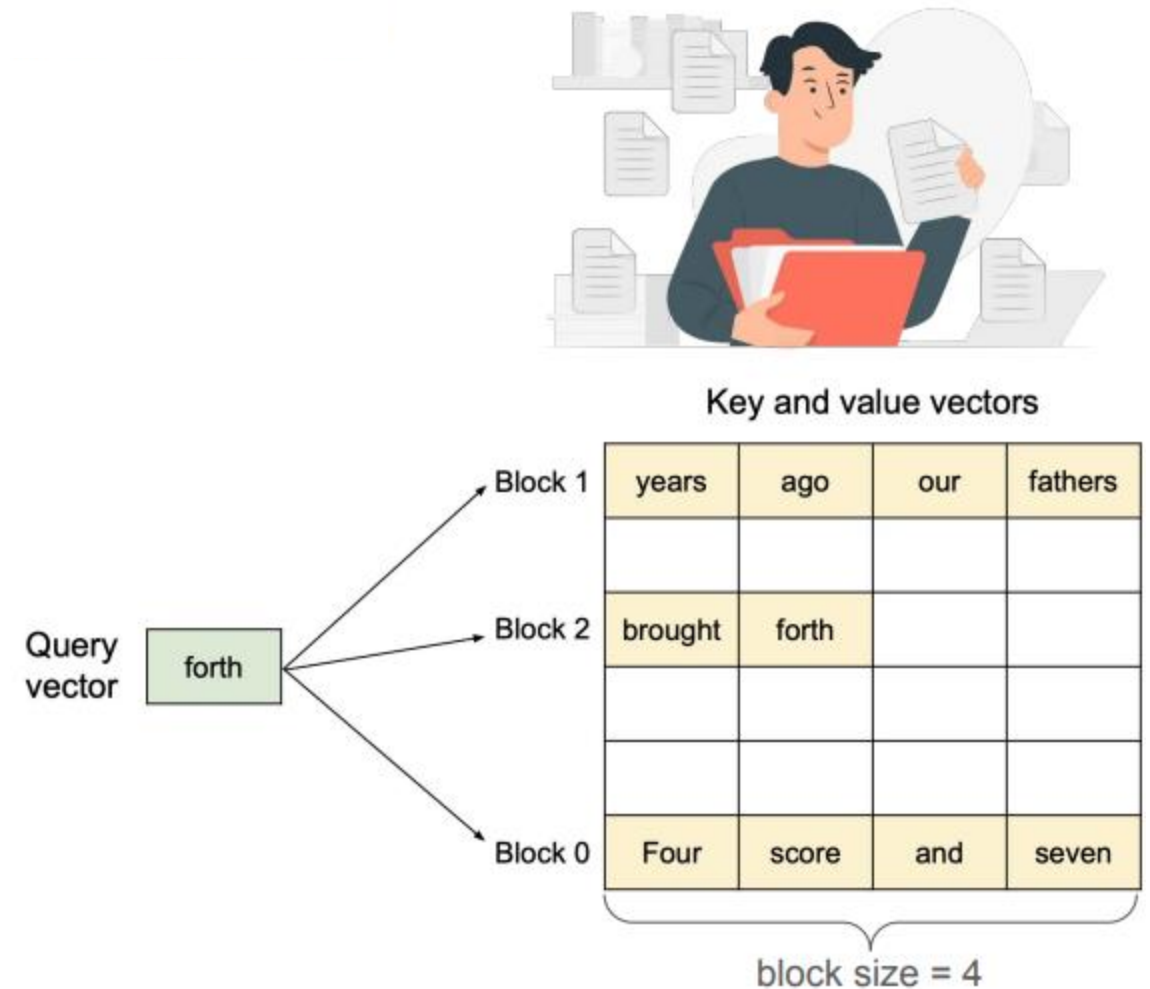


Key and value vectors

Block 1	years	ago	our	fathers
Block 2	brought	forth		
Block 0	Four	score	and	seven

block size = 4

- PagedAttention algorithm allows for storing attention key-value pairs in **non-contiguous blocks** in the memory
- Each KV block is a fixed-size contiguous chunk of memory that stores KV cache from left to right
- Attention for each new token is computed across all tokens in the non-contiguous KV block



Block Table For Managing KV Cache Blocks



0. Before generation.

Seq A Prompt: "Alan Turing is a computer scientist"
Completion: ""

Logical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
—	—
—	—
—	—
—	—

Physical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7				

Block Table For Managing KV Cache Blocks



1. Allocate space and store the prompt's KV cache.

Seq
A

Prompt: "Alan Turing is a computer scientist"
Completion: ""

Logical KV cache blocks

Block 0	Alan	Turing	is	a
Block 1	computer	scientist		
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
7	4
1	2
—	—
—	—

Physical KV cache blocks

Block 0				
Block 1	computer	scientist		
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7	Alan	Turing	is	a

Block Table For Managing KV Cache Blocks



2. Generated 1st token.

Seq A Prompt: "Alan Turing is a computer scientist"
Completion: "and"

Logical KV cache blocks

Block 0	Alan	Turing	is	a
Block 1	computer	scientist	and	
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
7	4
1	3
—	—
—	—

Physical KV cache blocks

Block 0				
Block 1	computer	scientist	and	
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7	Alan	Turing	is	a

Block Table For Managing KV Cache Blocks



3. Generated 2nd token.

Seq A Prompt: "Alan Turing is a computer scientist"
Completion: "and mathematician"

Logical KV cache blocks

Block 0	Alan	Turing	is	a
Block 1	computer	scientist	and	mathe- matician
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
7	4
1	4
—	—
—	—

Physical KV cache blocks

Block 0				
Block 1	computer	scientist	and	mathe- matician
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7	Alan	Turing	is	a

Block Table For Managing KV Cache Blocks



4. Generated 3rd token. Allocate new block.

Seq A

Prompt: “Alan Turing is a computer scientist”
Completion: “and mathematician renowned”

Logical KV cache blocks

Block 0	Alan	Turing	is	a
Block 1	computer	scientist	and	mathe- matician
Block 2	renowned			
Block 3				

Block table

Physical block no.	# Filled slots
7	4
1	4
3	1
—	—

Physical KV cache blocks

Block 0				
Block 1	computer	scientist	and	mathe- matician
Block 2	Allocated on demand			
Block 3	renowned			
Block 4				
Block 5				
Block 6				
Block 7	Alan	Turing	is	a

Block Table For Managing KV Cache Blocks



5. Generated 4th token.

Seq A Prompt: "Alan Turing is a computer scientist"
Completion: "and mathematician renowned for"

Logical KV cache blocks

Block 0	Alan	Turing	is	a
Block 1	computer	scientist	and	mathe- matician
Block 2	renowned	for		
Block 3				

Block table

Physical block no.	# Filled slots
7	4
1	4
3	2
-	-

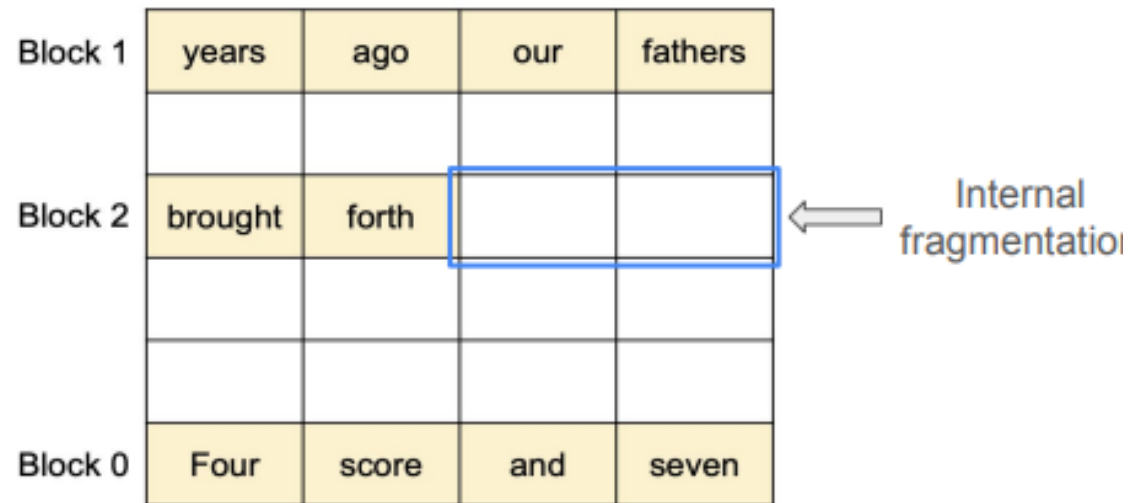
Physical KV cache blocks

Block 0				
Block 1	computer	scientist	and	mathe- matician
Block 2				
Block 3	renowned	for		
Block 4				
Block 5				
Block 6				
Block 7	Alan	Turing	is	a

Memory Efficiency of PagedAttention



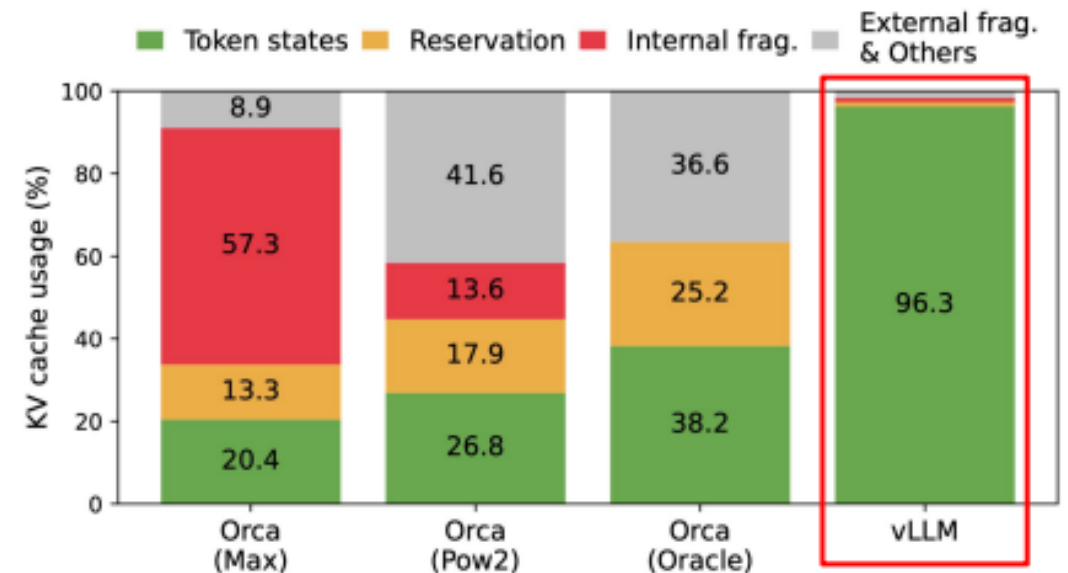
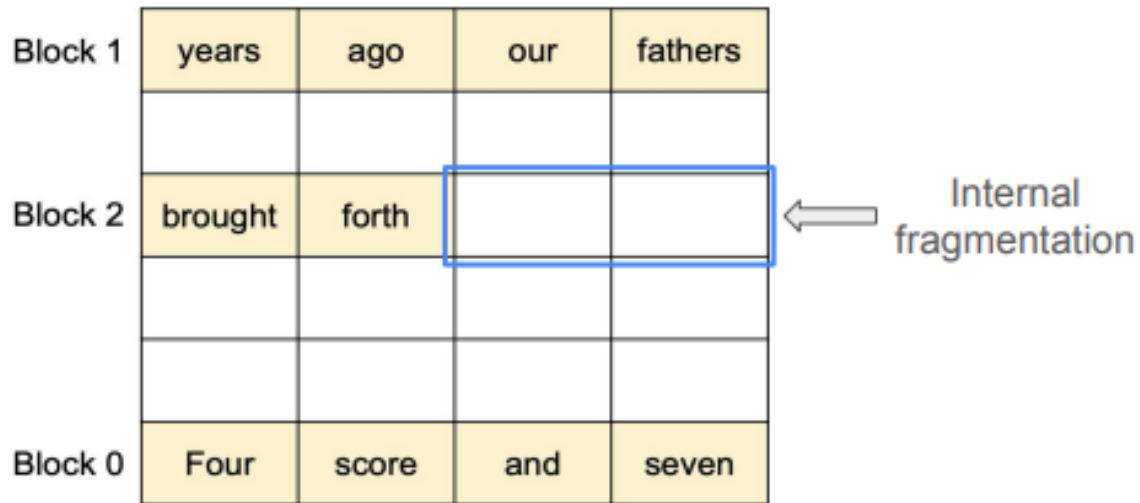
- Minimal internal fragmentation
 - # of wasted tokens per sequence < block size



Memory Efficiency of PagedAttention

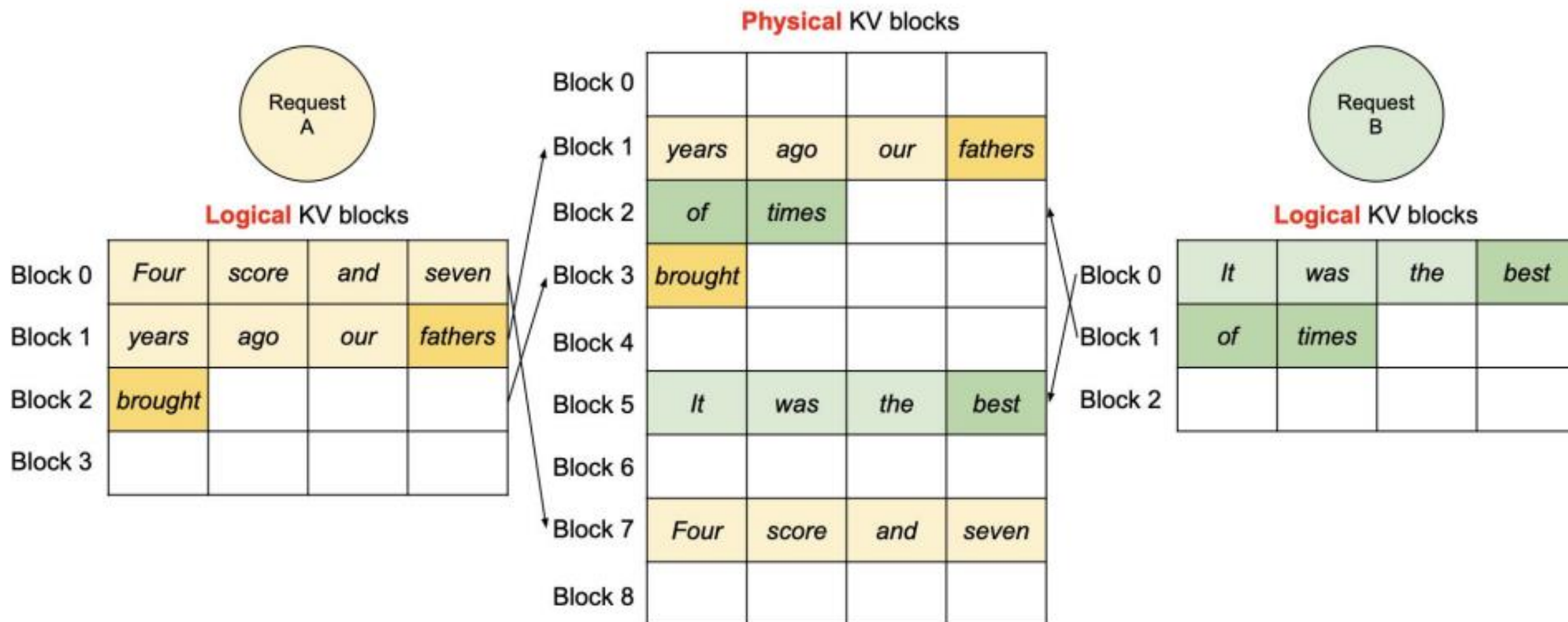


- Minimal internal fragmentation
 - # of wasted tokens per sequence < block size
- No external fragmentation (as we do not overclaim blocks)



- With PagedAttention, wasted KV cache space is < 4% (3-5x improved memory utilization)

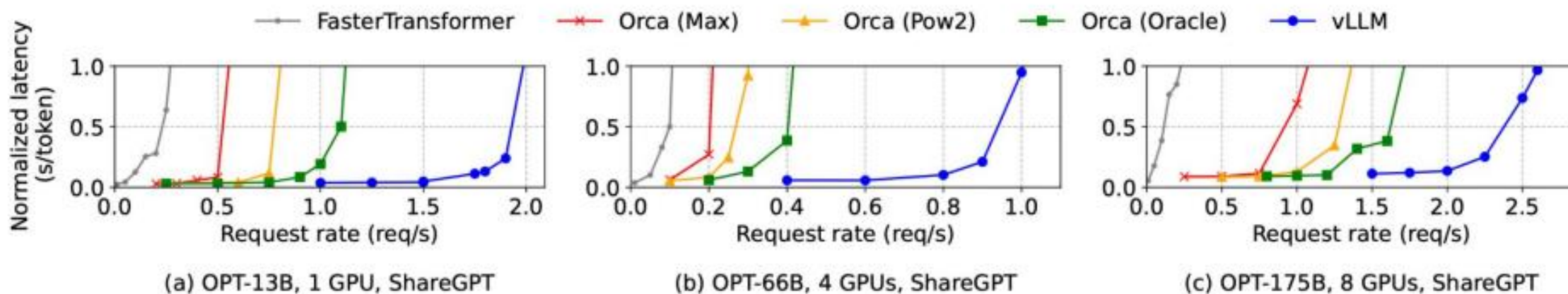
Handling Multiple Requests



vLLM Performance with Basic Generation



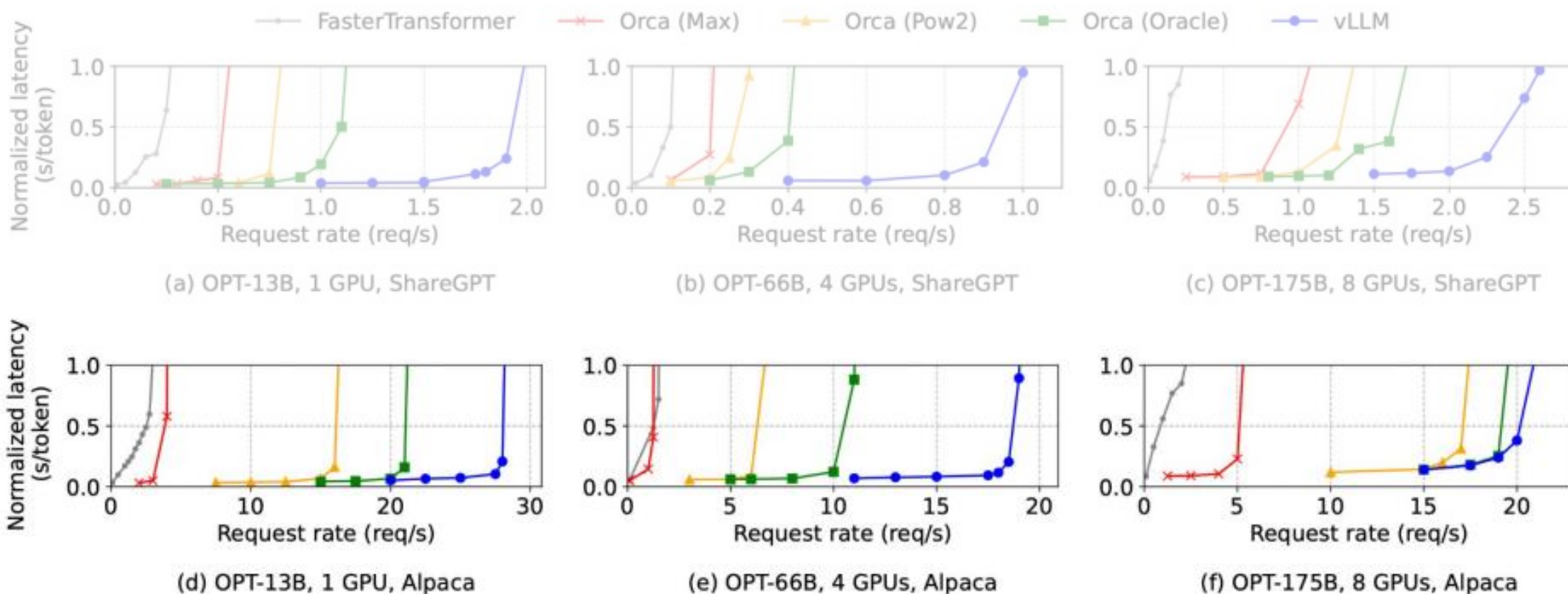
- one sample per request on three models and two datasets



vLLM Performance with Basic Generation



- one sample per request on three models and two datasets



- Reduces memory fragmentation with paging
- Reduces memory usage with KV block sharing
- Enables batching of more requests, increasing the throughput of LLM inference

4-page Mid-semester Report Due on Nov. 07

SGLang: Efficient Execution of Structured Language Model Programs

Lianmin Zheng^{2*} Liangsheng Yin³ Zhiqiang Xie¹ Chuyue Sun¹ Jeff Huang⁴
Cody Hao Yu⁵ Shiyi Cao² Christos Kozyrakis¹ Ion Stoica² Joseph E. Gonzalez²
Clark Barrett¹ Ying Sheng^{1*}

¹ Stanford University ² UC Berkeley ³ Shanghai Jiao Tong University
⁴ Texas A&M University ⁵ Independent Researcher

Allow prompts sharing the prefix input to use the same KV cache (~trie tree)

